Documentação da Sprint 1 — Bit or Byte

Projeto: BorB — Plataforma de Estudos Scrum

Período da Sprint 1: 28/09/2026 – 22/10/2026

Curso / Instituição: FATEC Jacareí – 1º DSM

Descrição do Trabalho

É comum que estudantes e profissionais iniciantes em TI encontrem dificuldades para fixar conceitos teóricos de metodologias ágeis (como Scrum e Kanban) e aplicá-los em cenários reais de desenvolvimento.

A falta de ferramentas práticas e interativas para mensurar o aprendizado gera uma lacuna no processo de formação. O BorB — Plataforma de Estudos Scrum visa resolver esse problema oferecendo uma trilha de simulados randômicos e cronometrados, acompanhada de uma Área de Estudos completa com 12 temas fundamentais, validando o conhecimento adquirido por meio da emissão de um certificado digital autêntico.

1. Equipe e Organização do Trabalho

1.1. Divisão de Papéis e Responsabilidades

- **Product Owner (PO):** Gestão e priorização do Product Backlog, organização do Kanban no GitHub Projects e integração final da branch `main`.

- **Scrum Master (SM):** Organização das demandas, facilitação das cerimônias do Scrum, garantia do progresso do Burndown e acompanhamento do time de desenvolvimento.

- **Equipe de Desenvolvimento (Dev Team):**
    - **Protótipo (Figma):** Nikolas Gabriel, Ryan Cesar, Sarah Cristiny.
    - **Front-End:** João Vitor Maximiano, Willian de Paula.
    - **Banco de Dados (PostgreSQL):** Matheus Oliveira, Pedro Lucas.
    - **Pesquisa de Conteúdo/Questões:** Luiz Felipe (inicialmente), expandido posteriormente para toda a equipe.

1.2. Gestão do Backlog e Kanban

- O fluxo de trabalho foi gerenciado via **GitHub Projects**.
- Na **Sprint Planning**, foi definida uma meta inicial de 12 tarefas.
- Ao longo da Sprint (no 22º dia), o escopo foi reajustado para 15 tarefas para acomodar novas necessidades (como a tela de recuperação de senha).

2. Funcionalidades Desenvolvidas e Implementadas (RNF06)

Durante a Sprint 1, foram desenvolvidas e consolidadas as seguintes entregas do sistema:

1. **Protótipo de Interface (Figma):**
    a. *Semanas 1 e 2:* Design das páginas Inicial, Login, Cadastro e Menu Principal.

b. *Semanas 3 e 4:* Finalização de todas as telas restantes da plataforma.

2. **Front-End (HTML5, CSS3, JavaScript puro):**

a. Implementação das telas codificadas em parceria direta com o time de prototipação.

3. **Banco de Dados Relacional (PostgreSQL):**

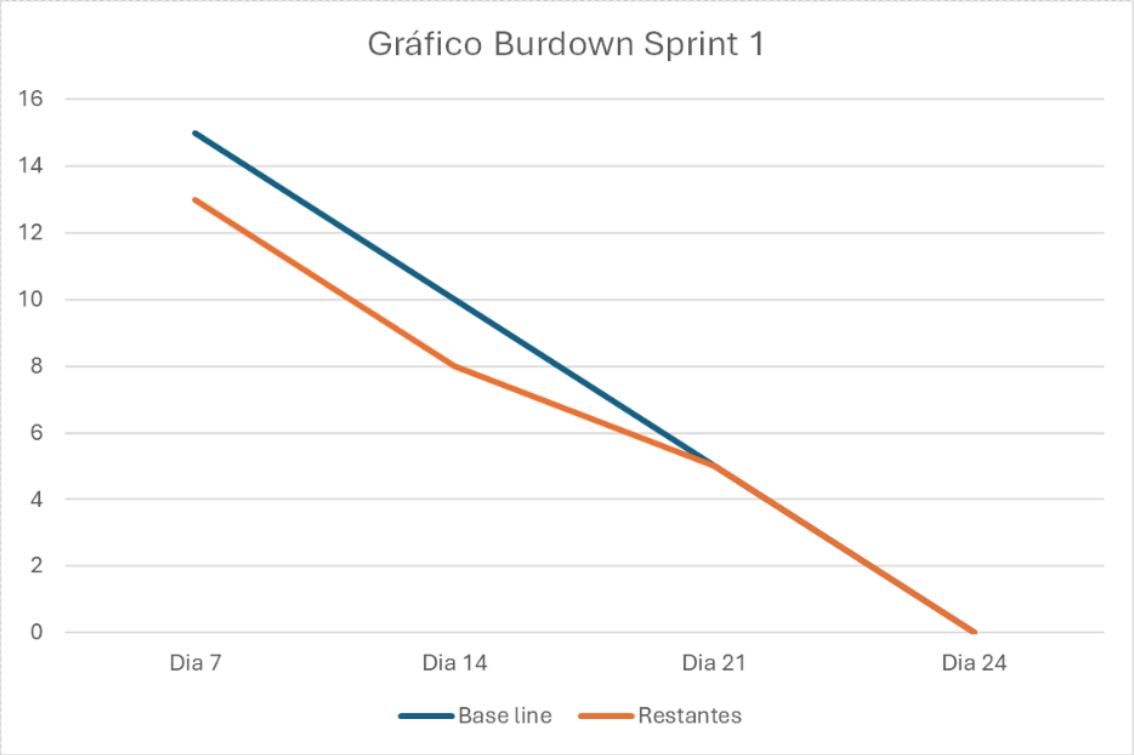
a. Criação e Modelagem: Estruturação dos comandos DDL para criação das tabelas e relacionamentos essenciais do sistema (usuários, temas, questões, alternativas e histórico).

b. Implementação e Conexão: Integração e configuração do driver de conexão na aplicação (back-end), permitindo consultas e operações seguras no banco de dados.

4. **Funcionalidades Adicionais:**

a. Fluxo de "Esqueci minha senha" com funcionalidade de recuperação de conta via e-mail.

3. Gráfico Burndown e Acompanhamento



- **Dia 0:** Planejamento inicial com 12 tarefas.
- **Dia 7:** 2 tarefas concluídas e movidas no Kanban.
- **Dia 14:** +2 tarefas concluídas (total de 4 entregas).
- **Dia 22:** Reajuste no backlog para 15 tarefas (10 concluídas até o momento).
- **Dias 22 a 24:** As 5 tarefas restantes ("em andamento") foram adiantadas e finalizadas com sucesso antes do término da Sprint.

4. Eventos e Práticas Scrum (RNF05)

- **Daily Scrum:** Realizada diariamente de forma remota via WhatsApp. O Scrum Master e o Product Owner monitoravam a evolução e reatribuíam demandas conforme necessário.

- **Planejamento Semanal:** O Scrum Master distribuiu e acompanhou as tarefas semanalmente para garantir o alinhamento com a meta do Burndown.

- **Critério de Pronto (Definition of Done — DoD):** Validação funcional, código integrado e aprovado nas branches correspondentes.

5. Versionamento e Engenharia de Software (Git & GitHub)

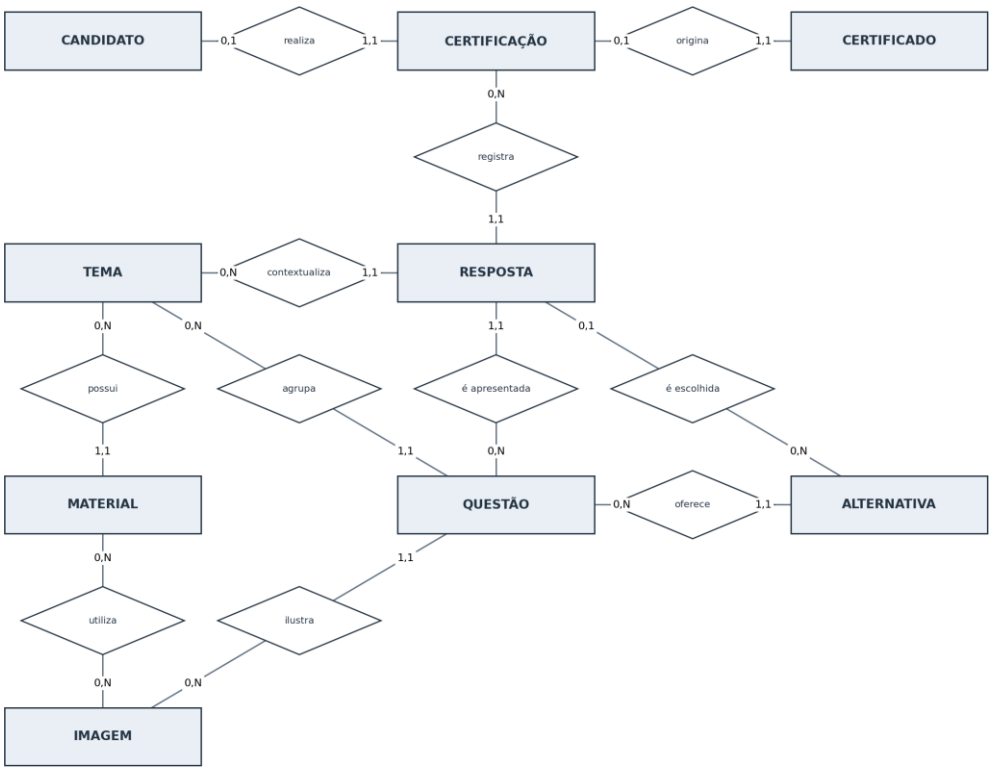
- **Estratégia de Branches:** Cada participante trabalhou em branches de funcionalidades específicas. Para manter a organização do repositório, todas as branches eram integradas na branch `DEVELOP`.

- **Consolidação:** Ao final da Sprint 1, após os testes e revisões, foi realizado o *merge* da branch `DEVELOP` para a branch `MAIN`.

6. Modelo de Dados (RNF06)

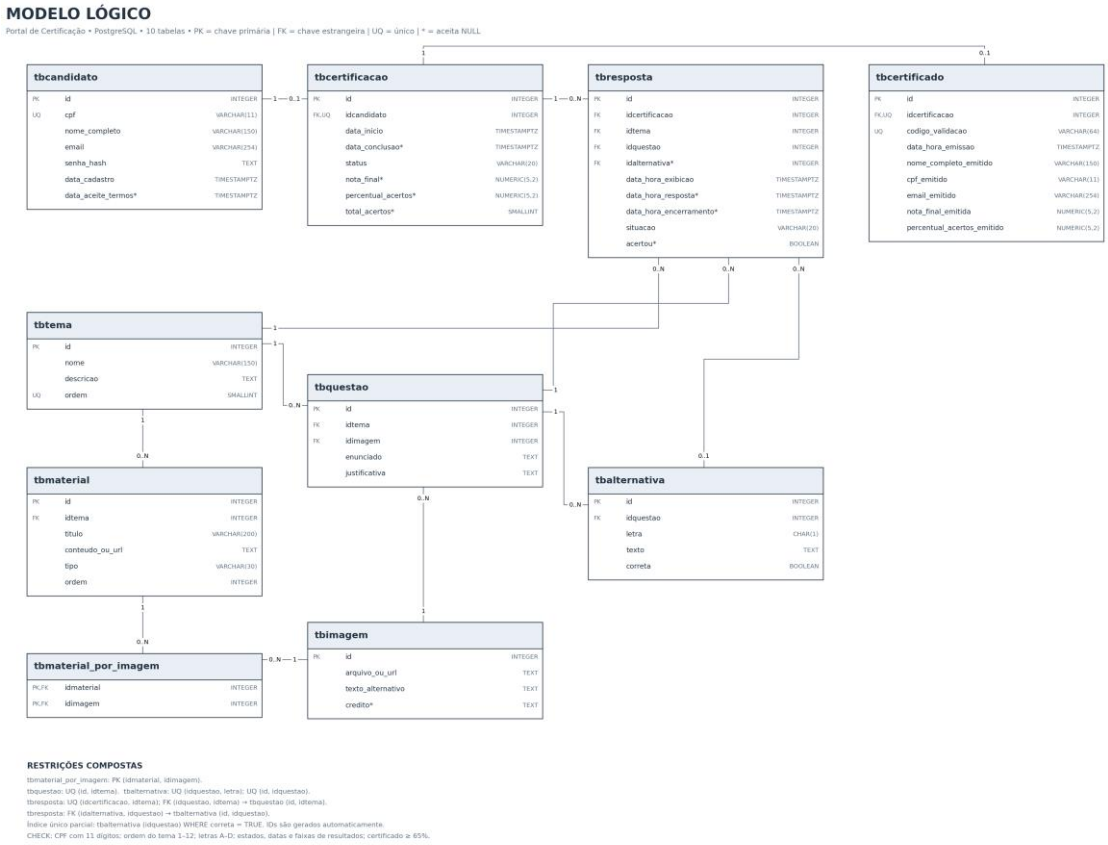
MODELO CONCEITUAL

Portal de Certificação • Entidades, relacionamentos e cardinalidades • Modelo atual



Cardinalidades: 0,1 = opcional único • 1,1 = obrigatório único • 0,N = zero ou vários

Cardinalidades indicam a participação de cada entidade e refletem a estrutura atual do banco.



- **Descrição:** Estrutura relacional do banco de dados desenvolvida em PostgreSQL contendo entidades como usuários, temas, questões, alternativas, histórico e certificados.

7. Documentação das APIs (Endpoints — RNF06)

Documentação dos endpoints implementados na Sprint 1 para comunicação entre o Front-End e o Back-End:

7.1. Autenticação e Usuários

POST /api/auth/register

-**Descrição**: Realiza o cadastro de um novo candidato na plataforma.

- **Body (JSON)**:

json { "nome": "Nome do Usuário", "cpf": "000.000.000-00", "email": "usuario@email.com", "senha": "sua_senha_segura" }

-**Respostas**:

* *201 Created:* Cadastro realizado com sucesso.

* *400 Bad Request*: Dados inválidos ou campos obrigatórios ausentes.

POST /api/auth/login

-**Descrição**: Autentica o usuário na plataforma utilizando credenciais de CPF e senha.

-**Body (JSON):**

json { "cpf": "000.000.000-00", "senha": "sua_senha_segura" }

-**Respostas**:

* *200 OK*: Login efetuado com sucesso (retorna o token de sessão/cookie).

* *401 Unauthorized*: Credenciais inválidas.

POST /api/auth/forgot-password

-**Descrição**: Solicita o processo de recuperação de senha. Como a aplicação opera em modo console para esta versão, o código gerado é enviado diretamente para o log do contêiner Docker da aplicação.

-**Body (JSON):**

json { "email": "usuario@email.com" }

-**Respostas**:

* *200 OK*: Solicitação processada e instruções/código gerados nos logs.

* *404 Not Found:* E-mail não encontrado na base de dados.

8. Instruções de Instalação e Execução (RNF06)

Pré-requisitos

-[Git](https://git-scm.com) instalado.

- [Docker Desktop](https://www.docker.com/products/docker-desktop/) em execução (inclui o Docker Compose).

Não é preciso instalar Node.js nem PostgreSQL, nem criar arquivos `.env`: o `docker-compose.yml` já traz toda a configuração.

Executando a Aplicação

1. Clonar o Repositório:

git clone https://github.com/bit-or-byte-devs/ABP-1-Portal-de-Certifcacao-Scrum.git

cd ABP-1-Portal-de-Certifcacao-Scrum

2. Subir os Contêineres (Docker Compose):

docker compose up -d –build

Dois serviços sobem: `db` (PostgreSQL 16, com o schema e os temas carregados automaticamente na primeira execução) e `app` (API Node.js que também serve o front-end). A primeira execução baixa as imagens e pode levar alguns minutos.

3. Acessar a Aplicação:

Abra o navegador em `http://localhost:3000/pages/index.html`

Comandos úteis

docker compose logs -f app # acompanhar o log (os códigos de recuperação de senha aparecem aqui)

docker compose down # parar os serviços (os dados do banco são mantidos)

docker compose down -v # parar e apagar o banco, para recomeçar do zero

Observações:

* A recuperação de senha roda em modo `console`: sem servidor SMTP, o código de verificação é impresso no log do serviço `app`.

* Se a porta 3000 já estiver em uso, altere o mapeamento `ports` do serviço `app` no `docker-compose.yml` (por exemplo, `"3001:3000"`).

* Os segredos do `docker-compose.yml` servem apenas para demonstração local. Para usar outros valores, defina `SESSION_SECRET` e `RECUPERACAO_SECRET` (32 caracteres ou mais) no ambiente.

9. Sprint Retrospective (Retrospectiva da Sprint 1)

A **Sprint Retrospective** foi realizada ao final da Sprint 1 com a participação de todos os membros da equipe (Product Owner, Scrum Master e Dev Team) para refletir sobre o processo de trabalho, identificar o que funcionou bem, o que pode ser melhorado e definir ações práticas de melhoria para a próxima Sprint.

9.1. O que funcionou bem? (Good Points)

- **Comunicação e Alinhamento:** Uso eficiente do WhatsApp para as *Dailies* diárias, permitindo rápido alinhamento entre o Scrum Master, Product Owner e o time de desenvolvimento.

- **Prototipagem Integrada:** Excelente sinergia entre a equipe de UI/UX (Figma) e os desenvolvedores Front-End, o que facilitou a codificação e fiel reprodução das telas.

- **Gestão e Entrega:** Flexibilidade e capacidade de reação do time, que conseguiu adiantar e concluir 100% das 15 tarefas (incluindo o reajuste do backlog) antes do encerramento da Sprint.

- **Conexão de Banco e Contêineres:** Sucesso na modelagem relacional no PostgreSQL e na orquestração do ambiente com Docker Compose.

9.1.1. DoR e DoD

- **DoR da Sprint 1 (Definition of Ready):**
  -**Protótipos no Figma:** Atende diretamente ao objetivo educacional de integrar a prototipagem no Figma e ao escopo inicial de design das interfaces.
  - **Modelagem de Dados PostgreSQL**: Cumpre a restrição de projeto que exige o uso de PostgreSQL com comandos DDL e DML para persistência, além da documentação mínima exigida (modelo de dados).
  -**Arquitetura em Containers (Docker Compose):** Atende rigorosamente ao requisito não funcional (RNF07) e à restrição de projeto (RP05), que determinam a orquestração de todos os serviços por meio do Docker Compose.

- **DoD da Sprint 1 (Definition of Done):**
  - **Scripts DDL/DML e Povoamento:** Valida a implementação inicial do banco de dados (tabelas essenciais como Usuários, Temas e Questões exigidas no escopo) rodando via Docker.
  -**Telas em HTML/CSS/JS nativo:** Respeita estritamente a restrição de projeto (RP01), que proíbe o uso de frameworks no front-end.

- **Endpoints de Cadastro e Autenticação (CPF e Senha)**: Atende diretamente aos requisitos funcionais RF03 e RF04, que determinam o cadastro e o login exclusivo por CPF e senha.

9.2. O que pode ser melhorado? (Pontos de Melhoria)

- **Ajustes de Escopo no Meio da Sprint:** O aumento do escopo de 12 para 15 tarefas no 22º dia mostrou que algumas necessidades técnicas/funcionais (como a tela de recuperação de senha) poderiam ter sido identificadas previamente na Sprint Planning.

- **Descentralização de Tarefas (Pesquisa de Conteúdo):** A tarefa de pesquisa de conteúdo iniciou centralizada em apenas um membro (Luiz) e precisou ser redistribuída ao longo da Sprint, gerando uma pequena sobrecarga inicial.

- **Documentação Técnica Inicial:** Faltaram detalhes mais refinados no guia de instalação e na especificação de variáveis de ambiente no repositório nas primeiras semanas.

- **Consolidação e Versionamento:** Fluxo de branches precisou ser melhor executado com a branch `DEVELOP` intermediando as entregas antes da fusão final na `MAIN` pelo PO.

9.3. Plano de Ação para a Sprint 2 (Action Items)

1. **Sprint Planning mais Detalhada:** Refinar com mais rigor as histórias de usuário e estimativas no dia 0 da próxima Sprint para evitar adiantamentos e reajustes significativos no meio do percurso.

2. **Distribuição Equilibrada de Pesquisas:** Planejar tarefas de levantamento de dados e conteúdo de forma distribuída desde o início da Sprint entre mais membros do time.

3. **Padronização da Documentação Técnica:** Manter atualizado o arquivo `README.md` e os arquivos de configuração (`.env.example`, scripts SQL de seed) no repositório GitHub para simplificar o setup de novos ambientes.

4. **Testes e Validação Antecipada:** Incluir a escrita de testes básicos e validação de endpoints no fluxo de Definition of Done (DoD) antes da integração na branch `DEVELOP`.